Intro to HTML, CSS, and JS

An Overview to Reading Web Code

Document Syllabus

This document is organized into two main sections. We begin with a simple introduction to HTML, CSS, and JavaScript, then move into the basic syntax we need to read and understand simple web code.

1. Introduction
    - What HTML, CSS, and JavaScript are
    - How the three languages work together
    - The body analogy: structure, appearance, and behavior
    - A simple visual example
2. Understanding the Basic Syntax
    - **HTML Syntax: Creating the Parts**
        - Definition of HTML syntax
        - Common HTML tags
        - A simple HTML activity example
        - Default HTML layout
        - Block-level and inline-level elements
        - The `<br>` tag
        - Tag attributes: adding extra details
    - **CSS Syntax: Styling the Parts**
        - Definition of CSS syntax
        - CSS selectors, properties, and values
        - Common CSS properties
        - How HTML class names connect to CSS selectors
        - Comments in HTML, CSS, and JavaScript
        - Adding CSS to a mini activity
        - Two ways to add CSS: inline CSS and style block CSS

Intro to HTML, CSS, and JavaScript

In today's digital world, creating interactive and visually appealing content for the web requires knowledge of three core technologies: HTML, CSS, and JavaScript. These three building blocks work together to form the structure, style, and interactivity of web pages and applications.

HTML
HyperText Markup Language

HTML creates the **structure** of web pages and interactive activities. It organizes the main parts of the page, such as text, images, buttons, questions, answer choices, links, and sections.

In simple terms, HTML is the **skeleton** of the page.

CSS
Cascading Style Sheets

CSS controls the **appearance** of a page or activity. It changes visual details such as colors, fonts, spacing, borders, layout, and animations.

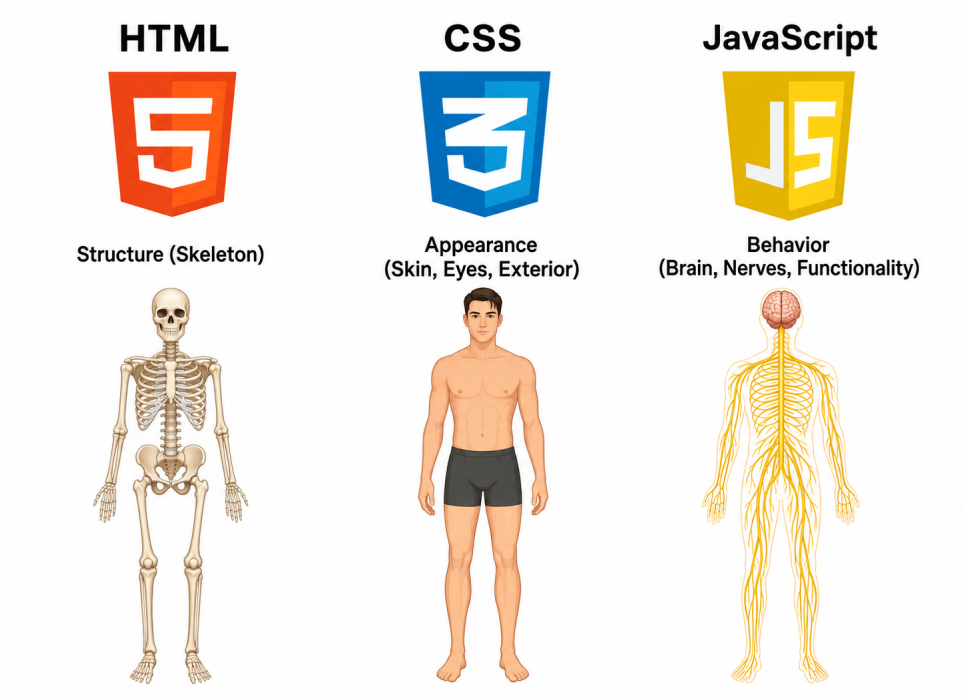
In simple terms, CSS is the **clothing** of the page.

JavaScript
Programming Language

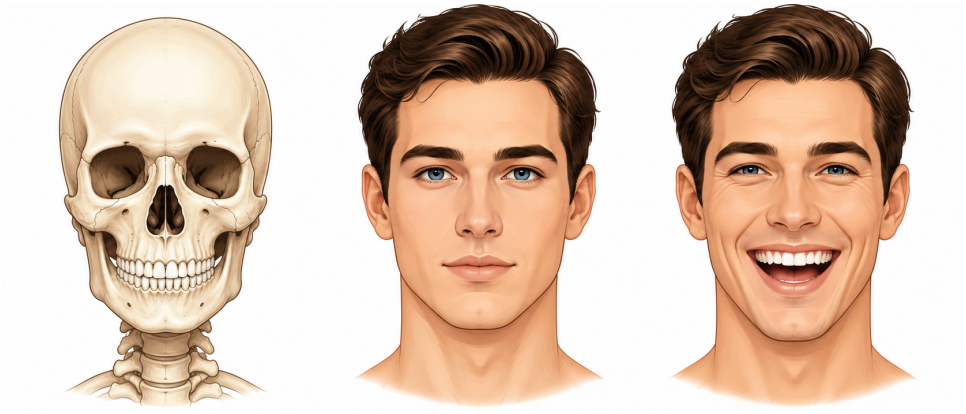
JavaScript adds **behavior and functionality** to web pages and activities. It makes the page respond when we click, type, select an answer, drag an item, or complete an action.

In simple terms, JavaScript is the **brain and nerves** of the page.

To visualize the concepts of HTML, CSS, and JavaScript, we can use the analogy of a human body.



Let's take a closer look with a simple example.



The close-up image shows the same idea in a smaller example.

HTML: The Structure

HTML creates the parts of the activity. In this example, it gives us the face, the eyes, and the mouth. These parts exist before we style them or make them move.

```
<div class="face">
  <div class="hair"></div>
  <div class="eye"></div>
  <div class="nose"></div>
  <div class="mouth"></div>
</div>
```

CSS: The Appearance

CSS changes how the parts look. It can give the face skin color, add hair color, make the eye blue, and shape the mouth.

```
.face {
  background-color: #ffd1b7;
  border-radius: 50%;
}

.hair {
  background-color: brown;
}

.eye {
  background-color: blue;
}

.mouth {
  border-radius: 50%;
}
```

JavaScript: The Behavior

In this example, a JavaScript function can change the mouth when we click the face or press a button, making it look like the face is laughing.

```
function laugh() {
  document
    .querySelector(".mouth")
    .classList.add("laugh");
}
```

Understanding the Basic Syntax

Before we start building small activities, it helps to understand the basic pattern behind the code. We do not need to memorize everything. Instead, we want to recognize what each part is doing so we can read AI-generated code, adjust it when needed, and explain what we want more clearly.

HTML Syntax: Creating the Parts

We can think of HTML as the part of the code that answers the question: **What do we want to place on the screen?**

For example, if we want a paragraph, a button, an image, a table, a choice, or a feedback box, HTML is what creates those pieces. CSS can style them later, and JavaScript can make them respond, but HTML gives us the actual elements to work with.

Most HTML elements are written using **tags**. A tag tells the browser what type of element we are creating.

```
<p>This is a paragraph.</p>
```

In this example, `<p>` opens the paragraph, and `</p>` closes it. The text between the opening and closing tags is **the content that appears on the page.**

```
<tag>Content that appears on the page</tag>
```

Common HTML Tags for Small Activities

- `<div>` creates a general block or container. We often use it to group parts of an activity.
- `<h1>`, `<h2>`, `<h3>` create headings with different sizes.
- `<p>` creates a paragraph of text.
- `<button>` creates a clickable button.
- `<input>` creates a field or choice we can interact with, such as a text box, checkbox, or radio button.
- `<label>` adds readable text beside an input. It helps us understand what each choice means.
- `<img>` adds an image.
- `<table>` creates a table for rows and columns.
- `<tr>` creates a table row.
- `<td>` creates a table cell.
- `<span>` targets a small piece of text inside a line.
- `<br>` creates a line break.

Here is a small example that uses a few common HTML tags together. This example only uses HTML, so it creates the content, but it does not style it yet.

HTML Code

```
<div class="page-wrapper">
  <div class="activity-card">
    <h2 class="activity-title">Quick Check</h2>
    <p class="activity-question">Which language creates the
     structure?</p>

    <input class="answer-input" type="radio" id="html" name=
     "language" value="html">
    <label class="answer-label" for="html">HTML</label><br>

    <input class="answer-input" type="radio" id="css" name=
     "language" value="css">
    <label class="answer-label" for="css">CSS</label><br>

    <input class="answer-input" type="radio" id="js" name=
     "language" value="js">
    <label class="answer-label" for=
     "js">JavaScript</label><br>

    <button class="custom-button">Submit</button>

    <p id="feedback" class="feedback-message"></p>
  </div>
</div>
```

Browser Output

Click the image to open the live HTML page.

In this example:

1. The `<div>` groups the activity into one block.
2. The `<h2>` creates the heading.
3. The `<p>` creates the question text.
4. The `<input>` creates each answer choice as a radio button.
5. The `<label>` adds the visible answer text beside each radio button.
6. The `<br>` moves each answer choice to a new line.
7. The `<button>` creates something we can click.
8. The `<p>` with id `feedback` creates an empty paragraph where we can later add feedback messages.

When the browser reads plain HTML, it displays the elements in their default layout.

- The browser reads the code in the order it is written, from **top to bottom**.
- Some elements, such as `<div>`, `<h2>`, and `<p>`, are **block-level elements**.
- Block-level elements usually start on a **new line**, so they appear under each other.
- Other elements, such as `<input>`, `<label>`, and `<button>`, are **inline-level elements**.
- Inline-level elements usually stay on the same line and render from **left to right**, as long as there is enough space.
- However, the `<br>` tag creates a line break, so it forces the next element to move to the next line.

In our example, each answer choice uses an `<input>` and a `<label>`. These two appear beside each other because they are inline-level elements. But after each label, we added `<br>`, so the next answer choice starts on a new line.

> **Attention Please!**
>
> The `<br>` tag is different from tags like `<p>` or `<div>` because it does not hold content. It only creates a line break. That is why we write `<br>` by itself, without a closing tag.

This simple activity appears at the top-left of the browser without any special spacing, color, or positioning. This is where CSS becomes useful. HTML creates the parts, but CSS helps us move them, style them, space them, and make the activity look more polished.

Tag Attributes: Adding Extra Details

Some HTML tags can include extra information called **attributes**. Attributes help describe how an element should behave, how it should be identified, or how it should connect to other parts of the activity.

We write attributes **inside the opening tag.**

```
<input class="answer-input" type="radio" id="html" name="language" value="html">
```

In this example, the `<input>` tag creates a choice. The attributes give that choice more meaning:

- `class="answer-input"` gives the input a class name we can style with CSS.

- `type="radio"` tells the browser this input is a radio button.
- `id="html"` gives this specific choice a unique identifier.
- `name="language"` groups the answer choices together.
- `value="html"` stores the value of the selected answer.

A simple way to remember this is: the **tag** creates the element, and the **attributes** give it extra details.

> **Attention Please!**
>
> Different HTML tags can have different attributes, and some attributes are valid for certain tags but not for others. For example, the `type` attribute is valid for `<input>` but not for `<p>`.

CSS Syntax: Styling the Parts

After HTML creates the parts of the activity, CSS helps us decide how those parts should look. We can think of CSS as the part of the code that answers the question: **How should this appear on the screen?**

CSS can control colors, spacing, borders, fonts, size, alignment, and layout. This is why plain HTML can look very simple at first, but the same HTML can look polished once CSS is added.

CSS usually follows this pattern:

```
selector {
  property: value;
}
```

1. The **selector** chooses the HTML element we want to style.
2. The **property** tells the browser what visual feature we want to change.
3. The **value** tells the browser how we want that feature to look.

A Small CSS Example

```
.activity-card {
  background-color: #ffefe9;
  border: 2px solid #ff4500;
  border-radius: 12px;
  padding: 16px;
  width: 300px;
}
```

In this example, `.activity-card` selects the HTML element with the class name `activity-card`. The lines inside the curly braces style that activity card.

- `background-color` changes the background color.
- `border` adds a border around the card.
- `border-radius` rounds the corners.
- `padding` adds space inside the card.
- `width` controls how wide the card is.

Attention Please!

The reason CSS knows which part to style is because the HTML has an attribute called `class`.

```
<div class="activity-card">
```

Then CSS uses that class name with a dot:

```
.activity-card {
  padding: 16px;
}
```

The dot before `.activity-card` means we are selecting a class. So, `class="activity-card"` in HTML connects to `.activity-card` in CSS.

Other selectors can target elements by their tag name, id, or other attributes. For now, we are using class names because they are a common and simple way to style specific parts of an activity.

Here is the basic idea:

```
.activity-card {} /* targets class="activity-card" */
#main-title {} /* targets id="main-title" */
button {} /* targets every <button> tag */
```

Common CSS Properties We May Use

- `color` changes the text color.
- `background-color` changes the background color.
- `font-size` changes the size of the text.
- `font-family` changes the style of the font.
- `padding` adds space inside an element.
- `margin` adds space outside an element.
- `border` adds a border around an element.
- `border-radius` rounds the corners.
- `text-align` aligns text to the left, center, or right.
- `display`: `flex` lets us arrange items more easily.
- `justify-content` controls horizontal alignment when we use flex.
- `align-items` controls vertical alignment when we use flex.
- `:hover` lets us change the style when we move the mouse over an element.

Attention Please!

The green lines in the CSS block are called **comments**. A comment is a note that does not get executed when the code runs. Its main purpose is to document the code and make it easier for teammates to understand.

Different languages write comments in different ways:

- HTML comment: `<!-- This is a comment -->`
- CSS comment: `/* This is a comment */`
- JavaScript comment: `// This is a comment`

Adding CSS to Our Mini Activity

Now we can return to the same HTML activity and add CSS to it. The HTML still creates the parts, but the CSS makes the activity card look more organized and polished.

HTML with Class Names

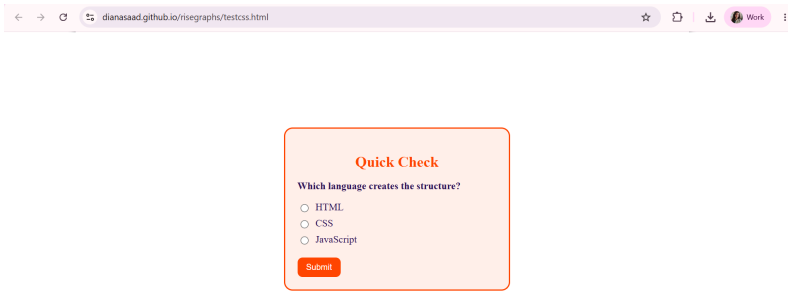
```
<div class="page-wrapper">
  <div class="activity-card">
    <h2 class="activity-title">Quick Check</h2>
    <p class="activity-question">Which language creates
     the structure?</p>

    <input class="answer-input" type="radio" id="html"
     name="language" value="html">
    <label class="answer-label" for=
     "html">HTML</label><br>

    <input class="answer-input" type="radio" id="css"
     name="language" value="css">
    <label class="answer-label" for=
     "css">CSS</label><br>

    <input class="answer-input" type="radio" id="js"
     name="language" value="js">
    <label class="answer-label" for=
     "js">JavaScript</label><br>

    <button class="custom-button">Submit</button>

    <p id="feedback" class="feedback-message"></p>
  </div>
</div>
```

CSS Style Block

```
.page-wrapper {
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
}

.activity-card {
  background-color: #ffefe9;
  border: 2px solid #ff4500;
  border-radius: 14px;
  padding: 20px;
  width: 320px;
  color: #2d1559;
}

.activity-title {
  color: #ff4500;
  text-align: center;
  margin-bottom: 10px;
}

.activity-question {
  color: #2d1559;
  font-weight: bold;
}

.answer-option {
  display: flex;
  align-items: center;
  gap: 8px;
  margin: 8px 0;
}

.answer-input {
  accent-color: #ff4500;
}

.answer-label {
  color: #2d1559;
}

.custom-button {
  background-color: #ff4500;
  color: white;
  border: none;
  border-radius: 8px;
  padding: 8px 14px;
  margin-top: 12px;
}
 This is the same class, but it is activated upon
      hovering
.custom-button:hover {
  background-color: #ff4500;
  opacity: 0.9;
}

.feedback-message {
  color: #ff4500;
}
```

In this version, we are still using the same HTML structure. The difference is that CSS now gives the card a background color, border, rounded corners, spacing, centered text, and a styled button. Let's see how this looks in the browser when we add the CSS.

Styled Browser Output



Click the image to open the live styled activity.

Two Ways to Add CSS

When we add CSS, we can write it in more than one way. For now, we will focus on two simple options: **inline CSS** and a **style block**.

- Inline CSS is written directly inside the HTML tag.
- A style block keeps the CSS in one place, which is usually cleaner when we are building a full activity.

Inline CSS

Inline CSS uses the `style` attribute inside the HTML tag.

```
<button class="custom-button" style="background-color:
      #ff4500; color: white;">
  Submit
</button>
```

This works for quick changes, but it can become harder to manage when we have many elements.

Style Block CSS

A style block keeps the CSS separate from the HTML structure.

```
<style>
.custom-button {
  background-color: #ff4500;
  color: white;
}
</style>

<button class="custom-button">Submit</button>
```

This is cleaner because the HTML creates the button, and the CSS controls how the button looks.

JavaScript Syntax: Adding Behavior

JavaScript is a coding language that runs in the browser. It helps the browser do more than just show text and images.

With JavaScript, we can target different parts of a web page and change what happens on the screen. For example, JavaScript can:

- change text after someone clicks a button,
- show or hide feedback,
- check if an answer is correct,
- change colors or classes,

- respond when someone types, clicks, drags, or selects something.

In simple terms, HTML creates the parts, CSS styles the parts, and JavaScript helps the page **react**.

Before we go into the code, it is important to understand what a **function** is in coding.

> **Attention Please!**
>
> A function is a set of instructions that we want to run together. Instead of writing the same instructions many times, we can place them inside one function and reuse that function when we need it.

A simple JavaScript function follows this syntax:

```
function functionName() {
  // function instructions go here
}
```

- The `function` keyword tells JavaScript that we are creating a reusable action.
- The `functionName` tells us what the action is called.
- The `()` are part of the function structure. They can also hold parameters, which are extra pieces of information we can pass to the function when we run it.
- The curly braces `{}` tell JavaScript where the function starts and ends. The code inside the curly braces is what gets executed when we run the function.

A Very Simple Function Example

Here is a very simple JavaScript function:

```
function sayHello() {
  alert("Hello!");
}
```

This function is called `sayHello`. When the function runs, it shows a small message that says `"Hello!"` on the browser screen.

To run the function, we call it by writing its name followed by parentheses:

```
sayHello();
```

So, the full idea is:

```
function sayHello() {
  alert("Hello!");
} // function definition

sayHello(); // function call (execution)
```

Built-in Browser Tools We May See

When we write JavaScript for a web activity, we often need it to find something on the page first. For example, JavaScript may need to find a button, an answer choice, or a feedback message.

To do this, it uses built-in browser tools such as `document`, `querySelector()`, and `getElementById()`.

Here are a few common ones we may see:

- `document` represents the web page that the browser is showing.
- `querySelector()` finds the first HTML element that matches a CSS selector.
- `getElementById()` finds one HTML element by its unique `id`.
- `getElementsByClassName()` finds HTML elements that share the same `class`.
- `value` lets JavaScript read the value stored inside an input.
- `checked` tells JavaScript whether a radio button or checkbox is selected.

Adding JavaScript to Our Mini Activity

Now that we have the HTML structure and CSS styling, we can add JavaScript to make the activity interactive.

What we want is the following behavior:

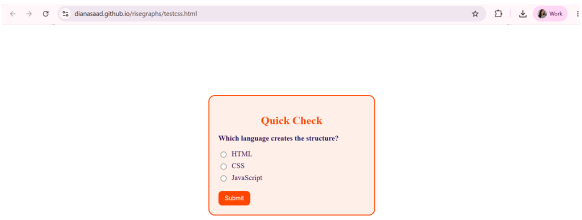
1. We select an answer choice by clicking on the radio button.
2. We click the submit button.
3. The JavaScript checks which answer we selected and compares it to the correct answer.
4. if the answer is correct, it shows a feedback message that says ”Correct!”.
5. Else, if the answer is incorrect, it shows a feedback message that says ”Incorrect. Try again!”.

> **Attention Please!**
>
> Before we write JavaScript, we should always know what behavior we want. This makes it easier to read the code, fix the code, or explain the code clearly in an AI prompt.

HTML and CSS Recap

```
<style>
  /* CSS code for styling the activity goes here. */
</style>


<div class="page-wrapper">
  <div class="activity-card">
    <h2 class="activity-title">Quick Check</h2>
    <p class="activity-question">Which language creates
     the structure?</p>

    <input class="answer-input" type="radio" id="html"
     name="language" value="html">
    <label class="answer-label" for=
     "html">HTML</label><br>

    <input class="answer-input" type="radio" id="css"
     name="language" value="css">
    <label class="answer-label" for=
     "css">CSS</label><br>

    <input class="answer-input" type="radio" id="js"
     name="language" value="js">
    <label class="answer-label" for=
     "js">JavaScript</label><br>

    <button class="custom-button">Submit</button>

    <p id="feedback" class="feedback-message"></p>
  </div>
</div>
```

Final HTML and CSS Output



At this stage, the activity has a question, answer choices, a submit button, and a feedback area.

However, the feedback area is still empty because we have not added the JavaScript logic yet.

Starting the Function

Now we can begin writing the function for our activity. Since the goal is to check the selected answer, we can name the function `checkAnswer`.

```
function checkAnswer() {
  // JavaScript instructions will go here.
}
```

At this point, the function is only an empty container. Next, we need to place the instructions inside it.

Reading the Selected Answer

The first instruction we need is to find the answer that we selected.

We can use `document` and `querySelector()` to search inside the HTML page.

```
let selectedAnswer = document.querySelector('input[name="language"]:checked');
```

This line means:

- Look inside the `document`, which means the web page.
- Find an `<input>` with the `name` attribute `(name="language")`.
- Only choose the input that is currently `checked`.
- Store that selected input inside a variable named `selectedAnswer`.

> **Attention Please!**
>
> The word `let` creates a variable. A variable is like a small labeled box where JavaScript can store information in memory and use it later.

Introducing If and Else

After JavaScript finds the selected answer, it needs to make a decision.

This is where we use an `if` and `else` block.

An `if` block checks a condition. If the condition is true, one set of instructions runs. If the condition is not true, the `else` block runs instead.

The basic structure looks like this:

```
if (// condition goes here) {
  // code runs when the condition is true
} else {
  // code runs when the condition is false
}
```

For our activity, the condition is:

```
selectedAnswer.value === "html"
```

This means: check whether the value of the selected answer is equal to `"html"`.

> **Attention Please!**
>
> In JavaScript, `===` is used to compare two values. It asks: are these two things exactly the same?

Writing the Full JavaScript Function

Now we can place the logic inside the function.

```
function checkAnswer() {
  let selectedAnswer = document.querySelector('input[name="language"]:checked');

  if (selectedAnswer.value === "html") {
    document.getElementById("feedback").textContent = "Correct! HTML creates the structure.";
  } else {
    document.getElementById("feedback").textContent = "Incorrect. Try again!";
  }
}
```

This function follows the logic we planned:

1. We find the selected radio button.
2. We read the selected answer's value.
3. We compare that value to `"html"`.
4. We show feedback based on the result.

How the Feedback Appears

The feedback message appears inside this paragraph from the HTML:

```
<p id="feedback"></p>
```

JavaScript finds this paragraph by using its `id`:

```
document.getElementById("feedback")
```

Then it changes the text inside the paragraph using `textContent`.

> **Important Note**
>
> JavaScript does not create the feedback paragraph here. The paragraph already exists in the HTML. JavaScript only finds it and changes the text inside it.

Triggering the JavaScript Function

The last piece is triggering the JavaScript function.

We want the `checkAnswer` function to run when we click the Submit button. To make that happen, we need to connect the button to the function.

In HTML, when we want something to happen after we click an element, we can use the `onclick` attribute. The `onclick` attribute tells the browser what JavaScript function should run when that element is clicked.

```
<button onclick="checkAnswer()">Submit</button>
```

In this example, the button says `onclick="checkAnswer()"`. This means: when we click the Submit button, the browser runs the `checkAnswer` function.

> **Attention Please!**
>
> The `onclick` attribute is commonly used with buttons because buttons are meant to be clicked. In our activity, it is the bridge between the HTML button and the JavaScript behavior.

Overall Activity Code

Now that we understand the main pieces separately, we can look at the full activity code together.

Javascript Script Block

Now that we understand the main pieces separately, we can look at the full activity code together.

```
<style>
.page-wrapper {
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
}

.activity-card {
  background-color: #ffefe9;
  border: 2px solid #ff4500;
  border-radius: 14px;
  padding: 20px;
  width: 320px;
  color: #2d1559;
}

.activity-title {
  color: #ff4500;
  text-align: center;
  margin-bottom: 10px;
}

.activity-question {
  color: #2d1559;
  font-weight: bold;
}

.answer-option {
  display: flex;
  align-items: center;
  gap: 8px;
  margin: 8px 0;
}

.answer-input {
  accent-color: #ff4500;
}

.answer-label {
  color: #2d1559;
}

.custom-button {
  background-color: #ff4500;
  color: white;
  border: none;
  border-radius: 8px;
  padding: 8px 14px;
  margin-top: 12px;
}

.custom-button:hover {
  background-color: #ff4500;
  opacity: 0.9;
}

.feedback-message {
  color: #ff4500;
}
}
</style>

<div class="page-wrapper">
  <div class="activity-card">
    <h2 class="activity-title">Quick Check</h2>
    <p class="activity-question">Which language creates the structure?</p>

    <input class="answer-input" type="radio" id="html" name="language" value="html">
    <label class="answer-label" for="html">HTML</label><br>

    <input class="answer-input" type="radio" id="css" name="language" value="css">
    <label class="answer-label" for="css">CSS</label><br>

    <input class="answer-input" type="radio" id="js" name="language" value="js">
    <label class="answer-label" for="js">JavaScript</label><br>

    <button class="custom-button">Submit</button>

    <p id="feedback" class="feedback-message"></p>
  </div>
</div>

<script>
  function checkAnswer() {
    let selectedAnswer = document.querySelector('input[name="language"]:checked');
```

```
    if (selectedAnswer.value === "html") {
      document.getElementById("feedback").textContent = "Correct! HTML creates the structure.";
    } else {
      document.getElementById("feedback").textContent = "Incorrect. Try again!";
    }
  }
</script>
```